



**INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**
Tocantins

Curso Superior de Tecnologia para Internet

Disciplina: Gerenciamento de Dados para Web **Professor:**

Fernando Jorge Ebrahim Lima e Silva

Aluno: Ulisses da Silva Bembem

Matrícula: 20222010350198

Exercício 06

1 - Qual das seguintes APIs para XML decompõe um documento XML em uma árvore de nós?

- ☐ a) SAX
- ☐ b) JAXB
- ☒ c) ~~DOM~~ DOM
- ☐ d) JAXP

2 - Em JAXB, qual anotação é usada para mapear a classe de mais alto nível para um elemento raiz no XML?

- ☐ a) @XmlElement
- ☒ b) ~~@XmlElement~~ @XmlRootElement
- ☐ c) @XmlType
- ☐ d) @XmlAccessorType

3 - Qual método da interface Node retorna o tipo de nó (por exemplo, Node.ELEMENT_NODE, Node.TEXT_NODE)?

- ☐ a) getNodeName()
- ☐ b) getNodeValue()
- ☒ c) ~~getNodeValue()~~ getNodeType()
- ☐ d) getParentNode()

4 - No contexto do JAXB, o processo de converter objetos Java em um documento XML é chamado de:

- ☐ a) Bind
- ☐ b) Unmarshal

- ☐ c) Parse
- ☒ d) Marshal

5 - Qual fábrica é usada para gerar instâncias de objetos SAXParser?

- ☐ a) DocumentBuilderFactory
- ☐ b) TransformerFactory
- ☒ c) SAXParserFactory
- ☐ d) JAXBContext

6 - Descreva a diferença fundamental entre as APIs SAX e DOM para interpretar documentos XML.

A API SAX decompõe o xml de maneira linear enquanto a DOM decompõe em formato de árvore, a primeira mais indicada para eficiência em memória quando processa grandes arquivos e a segunda apesar de menos eficiente no caso anterior possibilita a navegação e manipulação mais fácil, pois carrega todo o documento em uma árvore de nós.

7 - Explique o que é JAXB (Java Architecture for XML Binding) e quais são suas principais funcionalidades.

JAXB é uma API Java que facilita a conversão entre objetos Java e documentos XML. Suas principais funcionalidades são:

- **Marshal:** Converter objetos Java em XML.
- **Unmarshal** Converter XML em objetos Java.
- Mapeamento de classes Java para esquemas XML usando anotações como `@XmlRootElement`, `@XmlElement`.
- Suporte a validação de XML contra esquemas.

8 - Discorra sobre o processo de serialização de uma árvore DOM, mencionando as classes e métodos principais envolvidos.

A serialização de uma árvore DOM converte a estrutura em memória (objeto Document) em um fluxo XML, como um arquivo ou string. As principais classes e métodos envolvidos são:

- **TransformerFactory:** Cria um objeto Transformer via `TransformerFactory.newInstance().newTransformer()`.
- **Transformer:** Executa a serialização com o método `transform(Source, Result)`.
- **DOMSource:** Encapsula o Document como fonte de entrada.
- **StreamResult:** Define o destino da saída (ex.: arquivo ou System.out).
Exemplo: Um Transformer é configurado para serializar um Document em um arquivo XML, com opções como codificação e formatação (ex.: `setOutputProperty(OutputKeys.INDENT, "yes")`).

9 - Escreva um trecho de código Java que demonstre como criar um DocumentBuilder e fazer o parse de um arquivo XML chamado "meu_arquivo.xml" utilizando a API DOM. Inclua o tratamento básico de exceções.

```
public class ParseXML {
    public static void main(String[] args) {
        try {
            DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
            DocumentBuilder builder = factory.newDocumentBuilder();
            Document doc = builder.parse(new File("meu_arquivo.xml"));
            doc.getDocumentElement().normalize();

            System.out.println("Documento XML carregado com sucesso!");
        } catch (Exception e) {
            System.err.println("Erro ao processar o XML: " + e.getMessage());
        }
    }
}
```

10 - Considerando o JAXB, crie uma classe Java simples chamada Produto com os atributos nome (String) e preco (double). Adicione as anotações JAXB necessárias para que esta classe possa ser mapeada para um elemento XML chamado <produto> com os sub-elementos <nome> e <preco>. Demonstre como você realizaria o processo de de um objeto Produto para a saída padrão (System.out).

```

import javax.xml.bind.JAXBContext;
import javax.xml.bind.Marshaller;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;

@XmlRootElement(name = "produto")
public class Produto {
    private String nome;
    private double preco;

    public Produto() {}
    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }

    @XmlElement
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    @XmlElement
    public double getPreco() { return preco; }
    public void setPreco(double preco) { this.preco = preco; }

    public static void main(String[] args) {
        try {
            Produto produto = new Produto("Caneta", 2.50);
            JAXBContext context = JAXBContext.newInstance(Produto.class);
            Marshaller marshaller = context.createMarshaller();
            marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
            marshaller.marshal(produto, System.out);
        } catch (Exception e) {
            System.err.println("Erro ao realizar marshalling: " +
e.getMessage());
        }
    }
}

```

Saída esperada:

```

<?xml version="1.0" encoding="UTF-8" standalone="yes"?>

<produto>

    <nome>Caneta</nome>

    <preco>2.5</preco>

</produto>

```